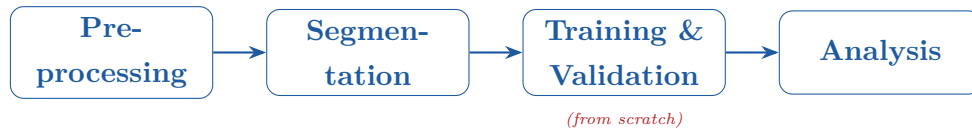

A Cutting-Edge End-to-End System for Skin Lesion Segmentation and Classification



Group 5

Deep Learning Course

Authors: Baraa Lazkani
Humam Yehia

Dataset: ISIC 2018 Task 3

Model: DenseNet-121 (pure NumPy, from scratch)

Date: May 2026

Contents

1	Introduction and Project Overview	5
1.1	Motivation	5
1.2	Original Contribution	5
1.3	Dataset: ISIC 2018 Task 3	6
1.4	Overall System Architecture	6
2	Project Directory Structure	7
2.1	Top-Level Layout	7
2.2	File Descriptions	8
2.2.1	Preprocessing & Segmentation	8
2.2.2	Training and Validation	9
2.2.3	Analysis	10
3	Phase 1: Data Preprocessing & Segmentation	11
3.1	Overview	11
3.2	Preprocessing Pipeline	11
3.2.1	Step Descriptions	12
3.3	Output Format	13
4	Phase 2: Training & Validation (From Scratch)	14
4.1	Configuration (configs/config.yaml)	14
4.2	Data Pipeline (src/dataset.py)	14
4.2.1	load_and_clean_csv	15
4.2.2	ImageTransform	15
4.2.3	DataLoader with Weighted Sampling	15
4.3	Model Architecture (src/model.py)	15
4.3.1	Low-Level Primitives	16
4.3.2	Individual Layers	16
4.3.3	DenseNet Building Blocks	17
4.4	Loss Function (src/losses.py)	18
4.4.1	Focal Loss	18
4.4.2	build_loss	18
4.5	Metrics (src/metrics.py)	18
4.6	Trainer (src/trainer.py)	19

4.6.1	Two-Phase Training	19
4.6.2	Key Methods	19
4.7	Utilities (<code>src/utis.py</code>)	20
4.7.1	Adam Optimizer	20
4.7.2	Cosine Annealing Scheduler	20
4.7.3	Other Utilities	20
4.8	Training Entry Point (<code>train.py</code>)	20
5	Phase 3: Analysis	22
5.1	Entry Point (<code>analyze.py</code>)	22
5.2	Key Functions	22
5.2.1	<code>load_artifacts</code> (<code>src/loader.py</code>)	22
5.2.2	<code>build_per_class_table</code> (<code>src/tables.py</code>)	22
5.2.3	<code>plot_training_curves</code> (<code>src/visualizer.py</code>)	23
5.2.4	<code>plot_confusion_matrix</code> (<code>src/visualizer.py</code>)	23
5.2.5	<code>plot_per_class_metric_comparison</code> (<code>src/visualizer.py</code>)	23
6	Results and Evaluation	24
6.1	Training Dynamics	24
6.2	Final Performance	24
6.3	Per-Class Performance	25
6.4	Confusion Matrix	26
6.5	Per-Class Metric Charts	26
6.6	Class Ranking by F1	27
7	Conclusion	29

List of Figures

1.1	End-to-end pipeline: from raw dermoscopy image to classification report	6
3.1	Data preprocessing and segmentation pipeline	11
4.1	DenseNet-121 macro-architecture (from scratch, NumPy). DB=Dense Block, T=Transition, GAP=Global Avg Pool.	17
4.2	Two-phase training strategy	19
4.3	Startup sequence in <code>train.py</code>	21
6.1	Training and validation loss over epochs	24
6.2	Validation metric curves over epochs (macro averages)	25
6.3	Confusion matrices at best epoch (epoch 34)	26
6.4	Per-class metrics at best epoch (all eight metrics)	26
6.5	Per-metric class comparison (green = best, red = worst)	28

List of Tables

1.1	ISIC 2018 Task 3 class distribution (training split)	6
4.1	Key training hyper-parameters	14
6.1	Macro-averaged metrics at best epoch (epoch 34)	25
6.2	Per-class metrics at best epoch	25
6.3	Class ranking by macro F1	27

Chapter 1

Introduction and Project Overview

1.1 Motivation

Skin cancer is among the most common cancers worldwide, and dermoscopy — the imaging of skin lesions through a dermatoscope — provides a non-invasive window into their structure. Accurate classification of dermoscopy images remains challenging even for trained dermatologists due to the subtle visual differences between benign and malignant lesions.

Computer-Aided Diagnosis (CAD) systems driven by deep learning have demonstrated remarkable potential in this domain. This project presents a complete end-to-end CAD pipeline for the **ISIC 2018 Task 3** benchmark, covering image enhancement, lesion segmentation, and multi-class classification.

1.2 Original Contribution

Scope of Original Work

The **Preprocessing & Segmentation** phase (Phase 1) was provided as part of the project framework. The **Training & Validation** phase (Phase 2) was designed and implemented **entirely from scratch** by the authors using pure **NumPy only** — no deep learning framework (PyTorch, TensorFlow, Keras, JAX, etc.) was used for the model, optimizer, loss, or data pipeline. This includes:

- A full DenseNet-121 forward and backward pass
- Custom convolution via `im2col/col2im`
- Batch Normalization, Dropout, Pooling layers
- Adam optimizer with bias correction
- Focal Loss with analytic gradient
- Cosine annealing and early stopping
- Weighted random sampling for class imbalance

The **Analysis** phase (Phase 3) post-processes the saved training logs and produces evaluation plots.

1.3 Dataset: ISIC 2018 Task 3

The International Skin Imaging Collaboration (ISIC) 2018 Task 3 provides labelled dermoscopy images across seven diagnostic categories. Table 1.1 shows the class distribution.

Table 1.1: ISIC 2018 Task 3 class distribution (training split)

Code	Full Name	Count	Description
MEL	Melanoma	1,113	Malignant melanocytic tumour
NV	Melanocytic Nevi	6,705	Common benign mole
BCC	Basal Cell Carcinoma	514	Most common skin cancer
AKIEC	Actinic Keratosis / Bowen’s Disease	327	Pre-malignant sun-damage lesion
BKL	Benign Keratosis-like Lesion	1,099	Seborrheic keratosis
DF	Dermatofibroma	115	Benign fibrous skin nodule
VASC	Vascular Lesions	142	Angioma, angiokeratoma, etc.
Total (usable)		9,835	

The dataset is heavily imbalanced: NV alone accounts for $\approx 68\%$ of samples, while DF and VASC together represent fewer than 3%. Addressing this imbalance is a key design concern in the training phase.

1.4 Overall System Architecture

Figure 1.1 shows the complete data and computation flow from a raw dermoscopy image to a classification decision.

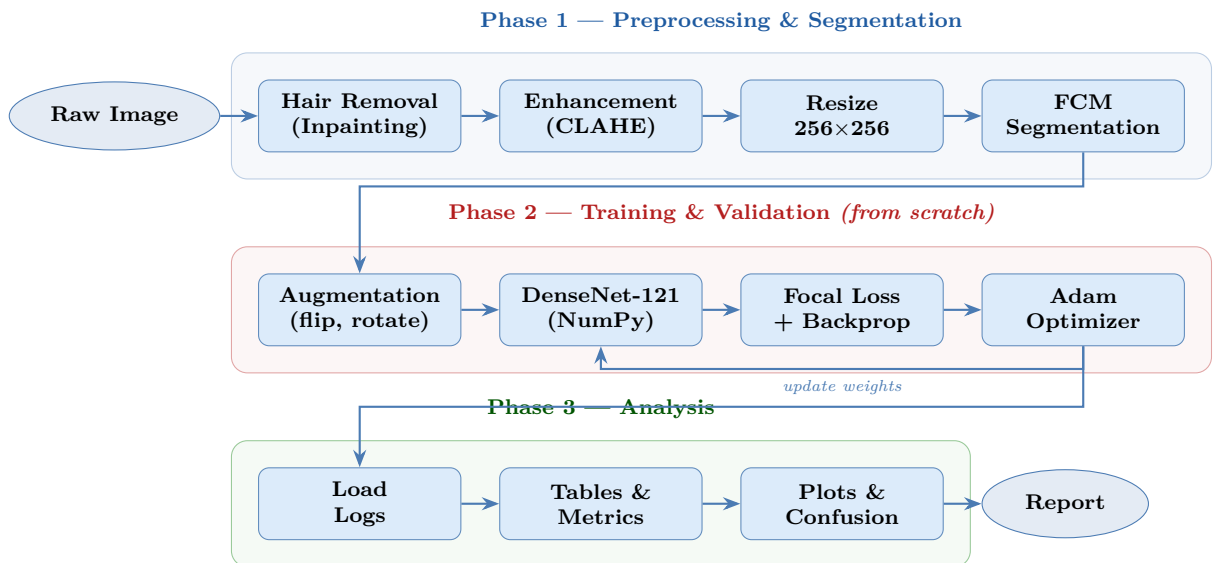


Figure 1.1: End-to-end pipeline: from raw dermoscopy image to classification report

Chapter 2

Project Directory Structure

2.1 Top-Level Layout

The repository is organised into three main directories — one per pipeline phase — plus a shared results folder produced by Phase 1 that is consumed by Phase 2.


```
Deep Learning Project/
|- Dara Preprocessing & Segmentation/
|   |- src/
|   |   '- 01_preprocessing_and_segmentation.ipynb
|   '- results/
|       |- ISIC2018_Task3_Training_GroundTruth.csv
|       |- ISIC2018_Task3_Test_GroundTruth.csv
|       |- images/training/ (segmented training JPEGs)
|       '- images/testing/ (segmented test JPEGs)
|
|- Training and Validation/
|   |- train.py
|   |- configs/config.yaml
|   |- src/
|   |   |- dataset.py
|   |   |- model.py
|   |   |- losses.py
|   |   |- metrics.py
|   |   |- trainer.py
|   |   '- utils.py
|   '- logs/
|       |- metrics.csv
|       |- best_metrics.json
|       |- best_confusion_matrix.npy
|       '- train.log
|
'- Analysis/
    |- analyze.py
    |- configs/analysis_config.yaml
    |- src/
    |   |- loader.py
    |   |- tables.py
    |   '- visualizer.py
    '- outputs/ (PNG plots, CSV/MD tables)
```

2.2 File Descriptions

2.2.1 Preprocessing & Segmentation

File / Directory	Purpose
src/01_preprocessing_and_segmentation.ipynb	Jupyter notebook containing the full image-processing pipeline: hair removal, CLAHE enhancement, Black Hat / Gaussian filtering, adaptive thresholding, inpainting, and Fuzzy C-Means (FCM) segmentation. Produces the segmented JPEG images consumed by Phase 2.
results/ISIC2018_Task3_Training-GroundTruth.csv	One-hot encoded ground-truth labels for the training set (columns: <code>image</code> , <code>MEL</code> , <code>NV</code> , <code>BCC</code> , <code>AKIEC</code> , <code>BKL</code> , <code>DF</code> , <code>VASC</code>).
results/ISIC2018_Task3_Test-GroundTruth.csv	Same format for the validation / test set.
results/images/training/	Segmented training images (filename pattern: <code>ISIC_XXXXXXX_segmented.jpg</code>).
results/images/testing/	Segmented test images (filename pattern: <code>ISIC_XXXXXXX_Test_Segmented.jpg</code>).

2.2.2 Training and Validation

File	Purpose
<code>train.py</code>	Entry point. Parses CLI arguments, loads the YAML config, seeds RNG, initialises the logger, device, dataloaders, model, loss, and optimizer, then calls <code>Trainer.fit()</code> .
<code>configs/config.yaml</code>	All hyper-parameters in one place: data paths, model settings, training schedule, augmentation probabilities, early-stopping criteria, and logging preferences.
<code>src/dataset.py</code>	<code>SkinLesionDataset</code> , <code>ImageTransform</code> (augmentation), and a custom <code>DataLoader</code> with weighted random sampling.
<code>src/model.py</code>	Complete DenseNet-121 architecture implemented with pure NumPy. Contains every layer from scratch: <code>Conv2D</code> , <code>BatchNorm2D</code> , <code>ReLU</code> , <code>Dropout</code> , <code>MaxPool2D</code> , <code>AvgPool2D</code> , <code>Linear</code> , <code>DenseLayer</code> , <code>DenseBlock</code> , <code>TransitionLayer</code> .
<code>src/losses.py</code>	<code>FocalLoss</code> — forward and analytic backward pass; <code>build_loss</code> factory that optionally adds per-class alpha weights.

File	Purpose
<code>src/metrics.py</code>	<code>compute_all_metrics</code> — computes per-class and macro Accuracy, Precision, Recall, Specificity, F1, Dice, Jaccard, and AUC-ROC from predictions and probabilities.
<code>src/trainer.py</code>	<code>Trainer</code> class: two-phase training loop (frozen backbone then full fine-tune), epoch-level validation, CSV logging, atomic model artifact saving, and early stopping.
<code>src/utils.py</code>	<code>AdamOptimizer</code> , <code>CosineAnnealingScheduler</code> , <code>ReduceOnPlateauScheduler</code> , <code>load_config</code> , <code>set_seed</code> , <code>setup_logger</code> , <code>get_device</code> .
<code>logs/metrics.csv</code>	Per-epoch training and validation metrics for the full run.
<code>logs/best_metrics.json</code>	Full per-class and macro metrics snapshot at the epoch with the best macro F1.
<code>logs/best_confusion_matrix.npy</code>	NumPy binary of the confusion matrix at the best epoch.
<code>logs/train.log</code>	Full timestamped training log (DEBUG level).

2.2.3 Analysis

File	Purpose
<code>analyze.py</code>	Entry point. Loads training artifacts from the <code>Training and Validation/logs/</code> directory and orchestrates all tabulation and plotting steps.
<code>configs/analysis_config.yaml</code>	Paths to the training logs and the output directory; decouples the analysis tool from the training directory layout.
<code>src/loader.py</code>	<code>load_artifacts</code> — reads <code>metrics.csv</code> , <code>best_metrics.json</code> , <code>final_metrics.json</code> , and <code>best_confusion_matrix.npy</code> ; returns them as a <code>SimpleNamespace</code> .
<code>src/tables.py</code>	<code>build_per_class_table</code> , <code>macro_summary_table</code> , <code>best_and_worst_classes</code> , <code>epoch_history_table</code> — export CSV and Markdown tables.
<code>src/visualizer.py</code>	<code>plot_training_curves</code> , <code>plot_per_class_metrics_bar</code> , <code>plot_confusion_matrix</code> , <code>plot_per_class_metric_comparison</code> — generate all PNG figures using Matplotlib and Seaborn.
<code>outputs/</code>	Contains all generated figures and tables (see Chapter 6).

Chapter 3

Phase 1: Data Preprocessing & Segmentation

Attribution

This phase was provided as part of the project framework and is not original work of the group. It is documented here for completeness and to explain the data format consumed by Phase 2.

3.1 Overview

The preprocessing notebook transforms raw dermoscopy JPEG images into clean, segmented lesion images that isolate the region of interest (ROI) from background skin and artefacts such as hair and air bubbles.

3.2 Preprocessing Pipeline

Figure 3.1 shows the sequential image-processing steps applied to each input image.

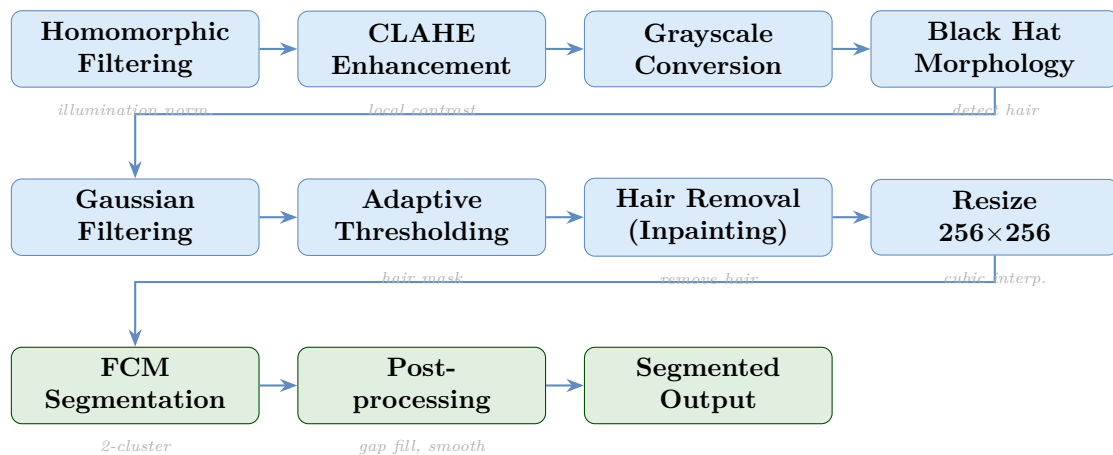


Figure 3.1: Data preprocessing and segmentation pipeline

3.2.1 Step Descriptions

Homomorphic Filtering

Normalises non-uniform illumination by separating and suppressing the low-frequency illumination component while preserving high-frequency reflectance detail. Operates in the log-frequency domain.

CLAHE (Contrast Limited Adaptive Histogram Equalization)

Applies local histogram equalisation with a clip limit to avoid noise amplification. Improves the visibility of fine lesion structures and colour gradients critical for segmentation.

Black Hat Morphological Filtering

Detects dark, elongated structures (hair fibres) against a brighter background by subtracting a morphologically closed image from the original. The resulting high-response regions form the hair detection mask.

Gaussian Filtering

Smooths the hair mask to reduce spurious detections before thresholding.

Adaptive Thresholding

Converts the smoothed hair response map into a binary hair mask, with the threshold adapting to local pixel neighbourhood statistics.

Inpainting (Hair Removal)

Uses the binary hair mask (dilated slightly to ensure full coverage) to fill over hair pixels using surrounding context, yielding a clean skin surface.

Resize to 256×256

Standardises image dimensions using cubic interpolation (bicubic), which preserves edge sharpness better than bilinear at this scale.

Fuzzy C-Means (FCM) Segmentation

Groups image pixels into $k = 2$ clusters (lesion ROI and background skin) using fuzzy membership. Parameters: fuzziness exponent $m = 0.001$, convergence tolerance $\varepsilon = 10^{-5}$, maximum 1,000 iterations. The ROI cluster forms the binary segmentation mask.

Post-processing

Fills small holes inside the segmentation mask (morphological closing) and smooths the mask boundary to remove jagged edges, improving the quality of the extracted region.

3.3 Output Format

Segmented images are saved as JPEG files with the naming conventions:

- **Training:** ISIC_XXXXXXX_segmented.jpg
- **Testing:** ISIC_XXXXXXX_Test_Segmented.jpg

Ground-truth labels are stored as one-hot-encoded CSV files alongside the images. The classifier in Phase 2 reads both the images and CSV files directly.

Chapter 4

Phase 2: Training & Validation (From Scratch)

This phase is the original contribution of the project. Every component — the model, the optimiser, the loss function, the data pipeline, and the training loop — was implemented using **pure NumPy with no deep learning library**.

4.1 Configuration (`configs/config.yaml`)

All hyper-parameters are centralised in a YAML file and loaded into a `SimpleNamespace` hierarchy at startup. Key settings:

Table 4.1: Key training hyper-parameters

Parameter	Value	Notes
Architecture	DenseNet-121	Built from scratch
Input size	224×224	Resized from 256×256
Dropout	0.6	Heavy regularisation for small dataset
Batch size	32	
Phase-1 epochs	8	Frozen backbone, classifier head only
Phase-1 LR	10^{-3}	High LR safe; only head updates
Phase-2 LR	3×10^{-5}	Low LR for full fine-tuning
LR scheduler	Cosine annealing	Applied in Phase 2
Loss	Focal ($\gamma = 2$)	Addresses class imbalance
Weight decay	10^{-3}	L2 regularisation
Weighted sampler	Enabled	Over-samples minority classes
Early stopping	patience=12	Monitors macro F1
Seed	42	

4.2 Data Pipeline (`src/dataset.py`)

4.2.1 load_and_clean_csv

Reads a ground-truth CSV, verifies that every referenced image file exists on disk, and derives an integer `label` column from the one-hot columns via `argmax`.

4.2.2 ImageTransform

Applies a configurable augmentation and normalisation pipeline. The transform is implemented as a callable class:

```

1 def __call__(self, pil_img: Image.Image) -> np.ndarray:
2     pil_img = pil_img.resize((self.size, self.size), Image.BILINEAR)
3     if self.augment and self.aug is not None:
4         pil_img = self._rotate(pil_img, self.aug.rotation_deg)
5     img = np.array(pil_img, dtype=np.float32) / 255.0
6     if self.augment and self.aug is not None:
7         img = self._hflip(img, self.aug.horizontal_flip)
8         img = self._vflip(img, self.aug.vertical_flip)
9         img = self._color_jitter(img, self.aug.color_jitter)
10    img = (img - self.MEAN) / self.STD    # ImageNet normalisation
11    return img.transpose(2, 0, 1).astype(np.float32)    # HWC -> CHW

```

Listing 4.1: Core transform call — augmentation and normalisation

Training augmentations applied: random horizontal flip ($p=0.5$), vertical flip ($p=0.5$), rotation up to $\pm 30^\circ$, and colour jitter (brightness, contrast, saturation, factor=0.2). Validation uses only resize and normalise.

4.2.3 DataLoader with Weighted Sampling

A custom `DataLoader` iterates over the dataset in batches. When `use_weighted_sampler` is enabled, each epoch samples indices *with replacement* using per-sample weights inversely proportional to class frequency:

$$w_i = \frac{1}{\text{count}(\text{class of sample } i)}$$

This effectively over-samples minority classes (DF, VASC) and under-samples the dominant class (NV), producing balanced mini-batches without discarding data.

4.3 Model Architecture (`src/model.py`)

4.3.1 Low-Level Primitives

`_im2col` / `_col2im`

Convolution is implemented via the *im2col* technique, which reshapes the input tensor into a 2-D matrix so that each convolution window becomes a row. The matrix product $\mathbf{W}_{\text{col}} \cdot \mathbf{X}_{\text{col}}^{\top}$ then computes all spatial positions simultaneously:

```

1 def _im2col(x, kernel_size, stride, padding):
2     N, C, H, W = x.shape
3     H_out = (H + 2*padding - kernel_size) // stride + 1
4     W_out = (W + 2*padding - kernel_size) // stride + 1
5     x_pad = np.pad(x, ((0,0),(0,0),(padding,padding),(padding,padding)
6     ))
7     s0, s1, s2, s3 = x_pad.strides
8     col = np.lib.stride_tricks.as_strided(
9         x_pad,
10        shape=(N, C, kernel_size, kernel_size, H_out, W_out),
11        strides=(s0, s1, s2, s3, s2*stride, s3*stride),
12    )
13     return col.transpose(0,4,5,1,2,3).reshape(N*H_out*W_out, -1).copy(
14        ), H_out, W_out

```

Listing 4.2: `im2col` using NumPy stride tricks

The gradient `_col2im` is the transpose operation, scattering gradient contributions back to each input position.

Layer Base Class

All layers inherit from a common `Layer` interface exposing `forward`, `backward`, `parameters`, `gradients`, and a `frozen` flag for selective gradient blocking.

4.3.2 Individual Layers

Conv2D

Stores weight tensor $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$ initialised with He initialisation. Forward: `im2col` + matrix multiply. Backward: weight gradient from outer product of gradient and cached column matrix; input gradient via `col2im`.

BatchNorm2D

Maintains running mean and variance for inference. During training, normalises over the (N, H, W) dimensions and scales by learned γ , β . Analytic backward computes $\partial L / \partial x$ via the chain rule through mean and variance.

ReLU

Caches a Boolean mask ($\mathbf{x} > 0$) during forward; gates the gradient in backward.

Dropout

Inverted dropout: scales the random binary mask by $1/(1-p)$ so the expected output magnitude is unchanged at test time. Disabled automatically when `training=False`.

MaxPool2D / AvgPool2D

Both use `im2col`. `MaxPool2D` caches the argmax index per pool window for gradient routing. `AvgPool2D` spreads the gradient uniformly across all positions.

Linear

Standard fully connected layer: $\mathbf{y} = \mathbf{x}\mathbf{W}^\top + \mathbf{b}$.

4.3.3 DenseNet Building Blocks

Figure 4.1 shows the DenseNet-121 macro-architecture.



Figure 4.1: DenseNet-121 macro-architecture (from scratch, NumPy). DB=Dense Block, T=Transition, GAP=Global Avg Pool.

DenseLayer

Each dense layer implements the *bottleneck* design: $\text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}(1 \times 1, 4k \text{ filters}) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}(1 \times 1, k \text{ filters})$, where $k = 32$ is the growth rate. The output is *concatenated* with the input along the channel axis, creating a direct gradient path to all preceding layers:

$$\mathbf{x}_\ell = [\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{\ell-1}]$$

DenseBlock

Stacks n `DenseLayer` objects sequentially. After n layers, the output has $C_{\text{in}} + n \cdot k$ channels. Backward pass reverses the layer order.

TransitionLayer

Between dense blocks, a transition layer performs $\text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}(1 \times 1, \text{compression}=0.5) \rightarrow \text{AvgPool}$ to halve spatial dimensions and reduce channel count.

DenseNet121 Top-Level Class

- **Stem:** Conv(7×7, 64, stride=2) + BN + ReLU + MaxPool(3×3, stride=2).
- **Body:** 4 Dense Blocks with 6, 12, 24, 16 layers respectively, interleaved with 3 Transition Layers.
- **Head:** Final BN + ReLU + Global Average Pooling + Dropout(0.6) + Linear(→7).
- **Backbone freeze / unfreeze:** Sets `frozen=True` on all layers except the final Linear to enable Phase-1 training.
- **Persistence:** `save(path)` and `load(path)` serialise / deserialise all weight arrays via `np.savez` keyed by layer name.

4.4 Loss Function (`src/losses.py`)

4.4.1 Focal Loss

Standard cross-entropy treats all samples equally. For heavily imbalanced datasets, easy majority-class samples dominate the gradient signal. Focal Loss addresses this by down-weighting well-classified examples:

$$\mathcal{L}_{\text{focal}} = -\alpha_t (1 - p_t)^\gamma \log p_t$$

where p_t is the model’s predicted probability for the true class, $\gamma = 2$ is the focusing parameter, and α_t is a per-class weight inversely proportional to class frequency.

The analytic backward pass computes:

$$\frac{\partial \mathcal{L}}{\partial z_j} = (1 - p_t)^\gamma (p_j - \mathbf{1}_{j=t}) - \gamma (1 - p_t)^{\gamma-1} p_t \log(p_t) \mathbf{1}_{j=t}$$

where z_j are the logits and $p_j = \text{softmax}(z)_j$.

4.4.2 `build_loss`

Factory function that reads the config to decide whether to use Focal Loss or weighted cross-entropy (Focal with $\gamma = 0$), and whether to use per-class α weights. When `use_weighted_sampler=True`, the sampler already balances batches so α is omitted.

4.5 Metrics (`src/metrics.py`)

`compute_all_metrics` accepts ground-truth labels, predicted labels, and predicted probabilities, then returns:

- **Per-class:** Accuracy, Precision, Recall, Specificity, F1 (= Dice), Jaccard Index, AUC-ROC
- **Macro averages** across all seven classes
- **Confusion matrix**

AUC-ROC is computed from scratch using a trapezoid integration over the sorted score ranking for each class in a one-vs-rest fashion.

4.6 Trainer (`src/trainer.py`)

4.6.1 Two-Phase Training

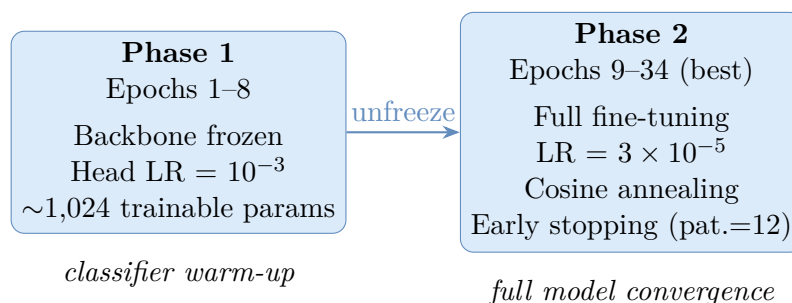


Figure 4.2: Two-phase training strategy

Phase 1 warms up the randomly initialised classifier head while keeping all convolution and batch-norm weights frozen. This prevents the random head from corrupting the backbone representations during early high-loss iterations.

Phase 2 unfreezes the entire model and fine-tunes with a much lower learning rate under cosine annealing, allowing all layers to specialise for the skin-lesion domain.

4.6.2 Key Methods

`train_one_epoch`

Iterates over all training batches, runs forward/backward through the model and loss, and calls `optimizer.step(grads)`. Uses a `tqdm` progress bar with live loss display.

`validate`

Sets the model to eval mode (disables dropout and uses running statistics for BN), collects all predictions and probabilities, then calls `compute_all_metrics` and logs the results table.

`_atomic_save`

Writes artifacts to a temporary file and atomically renames it, guaranteeing that a partial write never overwrites a valid checkpoint.

fit

Outer training loop. Orchestrates phase transitions, calls `train_one_epoch` and `validate` each epoch, logs to CSV, checks early stopping, and saves best/final artifacts.

4.7 Utilities (src/utls.py)

4.7.1 Adam Optimizer

Implements Adam with bias-corrected moment estimates:

$$\hat{m}_t = \frac{\beta_1 m_{t-1} + (1 - \beta_1) g_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{\beta_2 v_{t-1} + (1 - \beta_2) g_t^2}{1 - \beta_2^t}$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Default values: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Optional L2 weight decay is applied to the raw gradient before moment accumulation.

4.7.2 Cosine Annealing Scheduler

Decays the learning rate from its initial value $\eta_{\max}$ to $\eta_{\min} = 10^{-7}$ over $T_{\max}$ epochs:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\pi t}{T_{\max}}\right) \right)$$

4.7.3 Other Utilities

- `load_config`: YAML $\rightarrow$ nested `SimpleNamespace` for attribute-style access.
- `set_seed`: Seeds both `random` and `numpy.random` for reproducibility.
- `setup_logger`: Creates a dual-handler logger (file at DEBUG level, console at INFO level) with ISO timestamp formatting.

4.8 Training Entry Point (train.py)

`main()` wires all components together. The diagram in Figure 4.3 summarises the startup sequence.

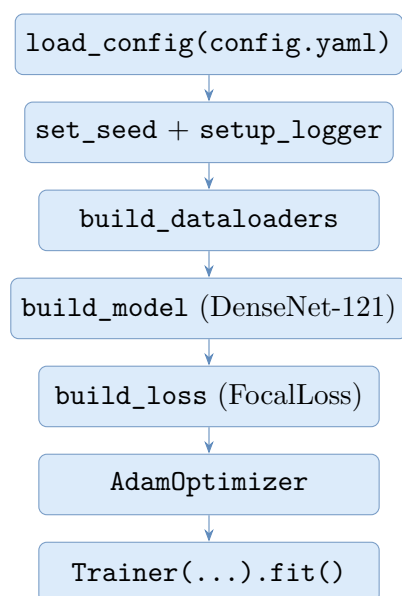


Figure 4.3: Startup sequence in `train.py`

Chapter 5

Phase 3: Analysis

After training completes, the `Analysis/` module reads the saved log artifacts and produces all evaluation tables and figures without re-running inference.

5.1 Entry Point (`analyze.py`)

Loads the YAML config, reads all artifacts via `loader.load_artifacts`, then calls:

1. `build_per_class_table` — per-class metrics CSV and Markdown
2. `macro_summary_table` — macro-averaged metrics CSV
3. `best_and_worst_classes` — ranked class summary text file
4. `epoch_history_table` — copy of `metrics.csv` to outputs
5. `plot_training_curves` — loss and metric curves
6. `plot_per_class_metrics_bar` — 8-panel bar chart
7. `plot_confusion_matrix` — count and normalised heatmaps
8. `plot_per_class_metric_comparison` — per-metric comparison with best/worst highlighting

5.2 Key Functions

5.2.1 `load_artifacts` (`src/loader.py`)

Validates that all required artifact paths exist before loading, raising a descriptive `FileNotFoundError` if training logs are absent. Returns a `SimpleNamespace` with fields `history` (`DataFrame`), `best` and `final` (dicts), and `confusion` (`NumPy` array).

5.2.2 `build_per_class_table` (`src/tables.py`)

Iterates over class names and metric keys, building a `pandas.DataFrame` that is saved as both a CSV and a Markdown table.

5.2.3 `plot_training_curves (src/visualizer.py)`

Generates two figures:

- Training loss vs. validation loss over all epochs.
- Macro F1, Accuracy, AUC-ROC, and Recall over all epochs.

5.2.4 `plot_confusion_matrix (src/visualizer.py)`

Produces two heatmaps from the saved `.npy` confusion matrix:

- **Count matrix:** raw prediction counts.
- **Normalised matrix:** row-normalised by true class totals (recall per cell).

5.2.5 `plot_per_class_metric_comparison (src/visualizer.py)`

Creates an 8-panel figure (one panel per metric). Within each panel, the best-performing class bar is coloured green and the worst red, making performance outliers immediately apparent.

Chapter 6

Results and Evaluation

6.1 Training Dynamics

Figure 6.1 shows the training and validation loss curves across all epochs. The phase transition at epoch 8 is visible as a brief spike in validation loss before the model begins to fully fine-tune.



Figure 6.1: Training and validation loss over epochs

Figure 6.2 shows the evolution of macro F1, Accuracy, AUC-ROC, and Recall on the validation set. The model converges to its best performance at **epoch 34**, after which early stopping terminates training.

6.2 Final Performance

Table 6.1 summarises the macro-averaged metrics at the best epoch.

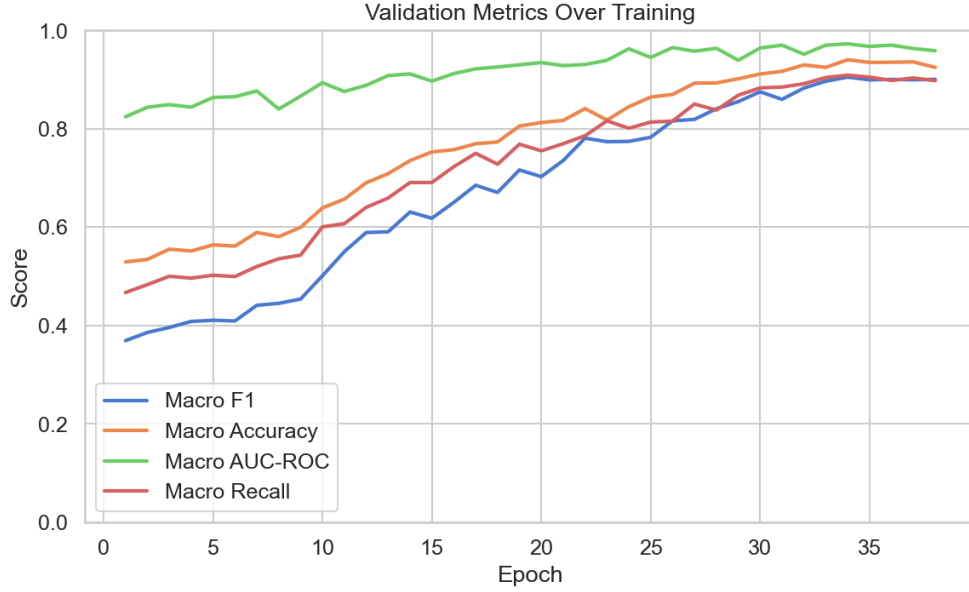


Figure 6.2: Validation metric curves over epochs (macro averages)

Table 6.1: Macro-averaged metrics at best epoch (epoch 34)

Accuracy	Precision	Recall	Specificity	F1	Dice	Jaccard	AUC-ROC
0.9405	0.9016	0.9091	0.9877	0.9052	0.9052	0.8283	0.9729

6.3 Per-Class Performance

Table 6.2 shows the full per-class breakdown. NV achieves the highest F1 (0.9732), reflecting its dominance in the training set even after rebalancing. DF achieves the lowest F1 (0.8769), consistent with it being the rarest class.

Table 6.2: Per-class metrics at best epoch

Class	Acc	Prec	Rec	Spec	F1	Dice	Jacc	AUC
MEL	0.9841	0.8991	0.8829	0.9921	0.8909	0.8909	0.8033	0.9681
NV	0.9689	0.9727	0.9738	0.9621	0.9732	0.9732	0.9479	0.9820
BCC	0.9881	0.9048	0.9223	0.9929	0.9135	0.9135	0.8407	0.9754
AKIEC	0.9835	0.8738	0.8824	0.9908	0.8780	0.8780	0.7826	0.9612
BKL	0.9709	0.9087	0.8924	0.9845	0.9005	0.9005	0.8189	0.9643
DF	0.9894	0.8769	0.8769	0.9945	0.8769	0.8769	0.7808	0.9701
VASC	0.9960	0.8750	0.9333	0.9973	0.9032	0.9032	0.8235	0.9889
Macro	0.9405	0.9016	0.9091	0.9877	0.9052	0.9052	0.8283	0.9729

6.4 Confusion Matrix

Figure 6.3 shows the normalised confusion matrix at the best epoch. The diagonal is uniformly high, indicating strong class separation. Minor confusion occurs between visually similar classes (MEL/NV and BKL/AKIEC).

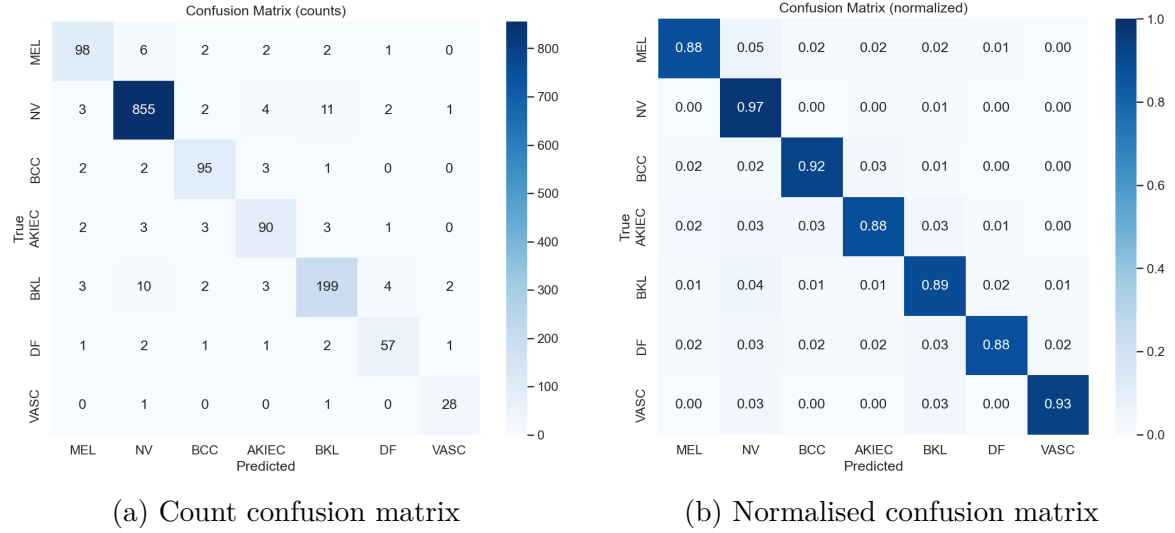


Figure 6.3: Confusion matrices at best epoch (epoch 34)

6.5 Per-Class Metric Charts

Figure 6.4 shows all eight metrics per class as grouped bar charts, and Figure 6.5 highlights the best (green) and worst (red) class for each metric.

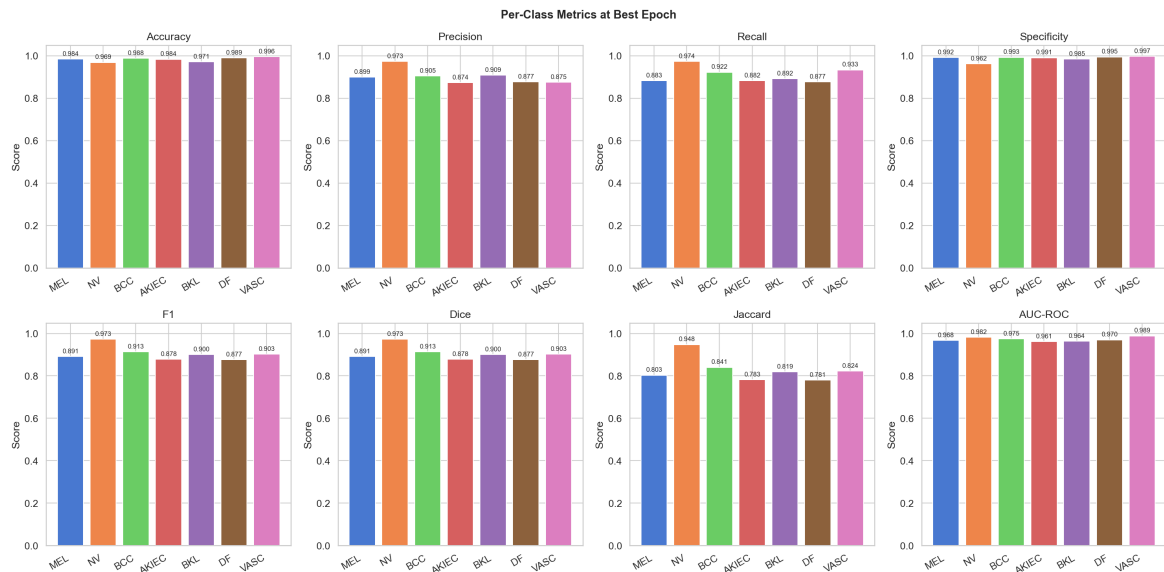


Figure 6.4: Per-class metrics at best epoch (all eight metrics)

6.6 Class Ranking by F1

Table 6.3: Class ranking by macro F1

Rank	Class	F1	
1	NV	0.9732	<i>Best</i>
2	BCC	0.9135	
3	VASC	0.9032	
4	BKL	0.9005	
5	MEL	0.8909	
6	AKIEC	0.8780	
7	DF	0.8769	<i>Worst</i>

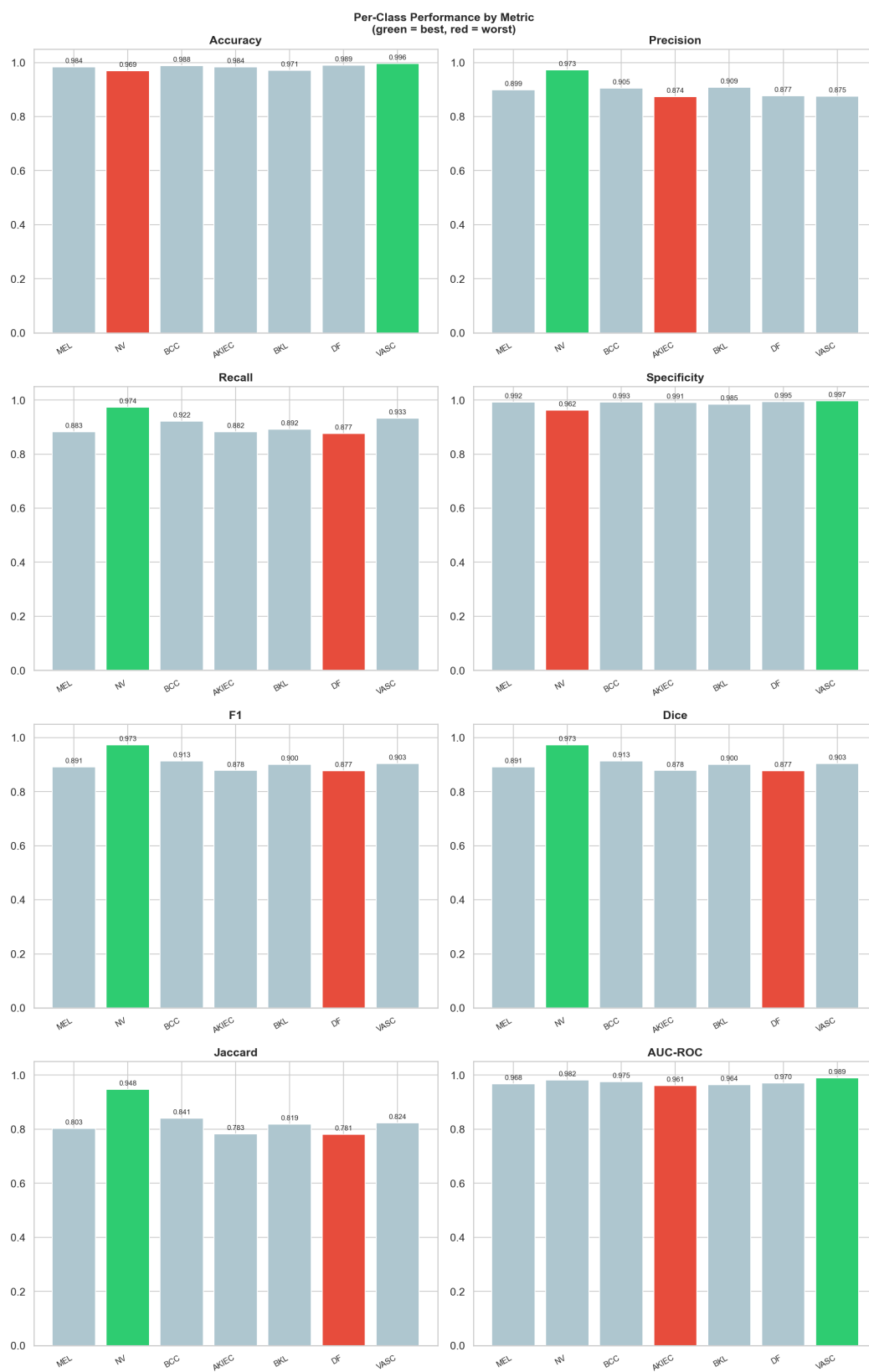


Figure 6.5: Per-metric class comparison (green = best, red = worst)

Chapter 7

Conclusion

This project demonstrates that a DenseNet-121 classifier built **entirely from scratch using NumPy** — with no deep learning framework — can achieve strong results on a real-world medical imaging benchmark. The system reaches a macro F1 of **0.9052** and an AUC-ROC of **0.9729** on the ISIC 2018 Task 3 seven-class skin lesion classification task.

Key design decisions that drove performance:

- **Weighted random sampling** and **focal loss** together address severe class imbalance without discarding data.
- **Two-phase training** (frozen backbone warm-up then full fine-tuning) prevents random-head destruction of feature representations in early epochs.
- **Cosine annealing** with a generous early-stopping patience of 12 epochs allows the optimiser to escape local minima while avoiding over-fitting.
- **Heavy dropout** (0.6) provides strong regularisation for a 7-million-parameter network trained on fewer than 10,000 images.

The preprocessing pipeline (Phase 1) contributes by supplying clean, hair-removed, contrast-enhanced images that let the classifier focus on true lesion morphology rather than artefacts.